

Applicazioni web

Upload di file

Come ricevere file da un form e gestirli in modo controllato.

Preparare il form

Per inviare file, il form deve usare `method="post"` e `enctype="multipart/form-data"` .

```
<form action="upload.php" method="post" enctype="multipart/form-data">
  <input type="file" name="documento">
  <button type="submit">Carica</button>
</form>
```

Senza `enctype` , il file non arriva correttamente a PHP.

Leggere il file con `$_FILES`

In `upload.php` :

```
<?php
$file = $_FILES["documento"] ?? null;

if ($file === null) {
    echo "Nessun file inviato";
    exit;
}

print_r($file);
```

`$_FILES` contiene informazioni come nome originale, tipo, dimensione, percorso temporaneo ed eventuale errore.

Controllare gli errori

```
<?php
if ($_FILES["documento"]["error"] !== UPLOAD_ERR_OK) {
    echo "Errore durante il caricamento";
    exit;
}
```

Controlla sempre l'errore prima di usare il file.

Controllare dimensione e tipo

```
<?php
$dimensioneMassima = 2 * 1024 * 1024;

if ($_FILES["documento"]["size"] > $dimensioneMassima) {
    echo "File troppo grande";
    exit;
}
```

Per il tipo, non fidarti solo del nome del file. Usa controlli server-side e accetta solo formati previsti.

Salvare il file

```
<?php
$cartella = __DIR__ . "/uploads";
$destinazione = $cartella . "/documento.pdf";

move_uploaded_file($_FILES["documento"]["tmp_name"], $destinazione);
```

`move_uploaded_file` sposta il file dalla cartella temporanea alla cartella scelta.

Attenzione: non salvare file caricati dagli utenti con nomi non controllati e non permettere l'esecuzione di file caricati. L'upload è una zona delicata della sicurezza web.

Creare la cartella di destinazione

Prima di spostare un file, assicurati che la cartella esista.

```
<?php
$cartella = __DIR__ . "/uploads";

if (!is_dir($cartella)) {
    mkdir($cartella);
}
```

`__DIR__` indica la cartella del file PHP corrente. In questo modo costruisci un percorso stabile.

Evitare nomi pericolosi

Il nome originale del file arriva dal computer dell'utente, quindi non va usato alla cieca.

```
<?php
$nomeSicuro = uniqid("file_", true) . ".pdf";
$destinazione = $cartella . "/" . $nomeSicuro;
```

`uniqid` crea un nome abbastanza unico per evitare sovrascritture accidentali. In un progetto reale controlleresti anche il tipo effettivo del file.

Checklist minima

Prima di salvare un file caricato, controlla sempre:

- che non ci siano errori di upload
- che la dimensione sia accettabile
- che il tipo sia tra quelli consentiti
- che il nome finale sia deciso dal server

- che la cartella di upload non esegua codice PHP

Form, GET e POST

Come ricevere dati inviati da un form HTML in PHP.

Cosa fa un form

Un form HTML permette a una persona di inserire dati e inviarli al server. PHP può leggere quei dati e usarli.

Un form con GET

```
<form action="cerca.php" method="get">
  <input type="text" name="q">
  <button type="submit">Cerca</button>
</form>
```

`action` indica quale file riceve i dati. `method` indica come vengono inviati.

In `cerca.php` puoi leggere il campo così:

```
<?php
$ricerca = $_GET["q"] ?? "";
echo "Hai cercato: " . htmlspecialchars($ricerca, ENT_QUOTES, 'UTF-8');
```

`$_GET` contiene i dati inviati con metodo GET. `?? ""` evita errori se il campo non esiste.

Quando usare GET

GET mette i dati nell'indirizzo della pagina. Va bene per ricerche e filtri.

Esempio:

```
cerca.php?q=php
```

Non usarlo per password o dati sensibili.

Un form con POST

```
<form action="contatto.php" method="post">
  <input type="text" name="nome">
  <textarea name="messaggio"></textarea>
  <button type="submit">Invia</button>
</form>
```

In `contatto.php` :

```
<?php
$nome = $_POST["nome"] ?? "";
$messaggio = $_POST["messaggio"] ?? "";

echo "Grazie, " . htmlspecialchars($nome, ENT_QUOTES, 'UTF-8');
```

`$_POST` contiene i dati inviati con metodo POST.

Validare sempre

Non fidarti mai dei dati ricevuti da un form. Controlla che ci siano, che abbiano il formato giusto e che non siano troppo lunghi.

```
<?php
if ($nome === "") {
    echo "Il nome e obbligatorio";
}
```

La validazione rende il programma piu sicuro e piu gentile con chi lo usa.

Mantenere il valore dopo l'invio

Quando un form contiene errori, è gentile mostrare di nuovo i valori inseriti.

```
<?php
$nome = $_POST["nome"] ?? "";
?>

<form method="post">
    <input type="text" name="nome" value="<?= htmlspecialchars($nome,
ENT_QUOTES, 'UTF-8') ?>">
    <button type="submit">Invia</button>
</form>
```

Se il form viene inviato, `$nome` contiene il valore ricevuto. `htmlspecialchars` lo rende sicuro prima di rimetterlo nell'HTML.

Controllare il metodo della richiesta

Una pagina può essere aperta normalmente o ricevere un invio dal form. Puoi distinguere i due casi con `$_SERVER["REQUEST_METHOD"]`.

```
<?php
if ($_SERVER["REQUEST_METHOD"] === "POST") {
    $nome = $_POST["nome"] ?? "";

    if ($nome === "") {
        echo "Il nome è obbligatorio";
    }
}
```

In questo modo validi i dati solo quando il form è stato inviato.

Regola pratica

Usa GET quando il risultato può stare nell'indirizzo, come una ricerca. Usa POST quando stai inviando dati che cambiano qualcosa o che non vuoi mostrare nell'URL.

PHP e HTML

Come mescolare PHP e HTML per generare pagine dinamiche.

Due ruoli diversi

HTML descrive la pagina. PHP prepara dati e pezzi dinamici della pagina.

```
<?php
$titolo = "Benvenuto";
?>

<h1><?php echo $titolo; ?></h1>
```

PHP calcola il valore. HTML lo mostra.

Echo dentro HTML

Per stampare un valore puoi usare `echo` .

```
<?php
$nome = "Luca";
?>

<p>Ciao, <?php echo $nome; ?>!</p>
```

Esiste anche una forma breve:

```
<p>Ciao, <?= $nome ?>!</p>
```

`<?= $nome ?>` significa: stampa questo valore.

Separare logica e markup

Cerca di preparare i dati prima dell'HTML.

```
<?php
$utente = [
    "nome" => "Sara",
    "ruolo" => "amministratrice",
];
?>

<h1><?= $utente["nome"] ?></h1>
<p>Ruolo: <?= $utente["ruolo"] ?></p>
```

Questo rende la pagina più leggibile: prima organizziamo i dati, poi li mostriamo.

Scorrere dati in una pagina

```
<?php
$prodotti = ["Penna", "Quaderno", "Zaino"];
?>

<ul>
    <?php foreach ($prodotti as $prodotto): ?>
        <li><?= $prodotto ?></li>
    <?php endforeach; ?>
</ul>
```

La sintassi con `endforeach` è comoda nei template HTML per evitare troppe parentesi graffe.

Proteggere il testo

Quando stampi dati che arrivano da utenti, usa `htmlspecialchars`.

```
<p><?= htmlspecialchars($nome, ENT_QUOTES, 'UTF-8') ?></p>
```


E una piccola abitudine che evita molti problemi di sicurezza.

Alternare PHP e HTML senza confusione

Quando lavori in una pagina mista, prova a pensare in due fasi:

1. PHP prepara i dati
2. HTML mostra i dati

```
<?php
$titolo = "Catalogo";
$numeroProdotti = 3;
?>

<h1><?= htmlspecialchars($titolo, ENT_QUOTES, 'UTF-8') ?></h1>
<p>Prodotti disponibili: <?= $numeroProdotti ?></p>
```

Questo ordine evita di infilare calcoli lunghi in mezzo al markup.

Condizioni dentro una pagina

Puoi mostrare HTML diverso in base a una condizione.

```
<?php
$utenteLoggato = false;
?>

<?php if ($utenteLoggato): ?>
    <p>Benvenuto nella tua area personale.</p>
<?php else: ?>
    <p>Accedi per continuare.</p>
<?php endif; ?>
```

Questa forma è leggibile nei template perché separa bene il codice PHP dai pezzi HTML.

Errore comune: fare troppa logica nel template

Se dentro l'HTML inizi ad avere molti calcoli, query al database o controlli lunghi, la pagina diventa difficile da mantenere. Prepara prima i dati in PHP, poi usa il template solo per mostrarli.

Sessioni e cookie

Come conservare informazioni tra una richiesta web e l'altra.

Il problema della memoria nel web

Ogni richiesta web è separata dalle altre. Se apri una pagina e poi un'altra, PHP non ricorda automaticamente chi sei.

Sessioni e cookie servono a conservare informazioni tra una richiesta e la successiva.

Cookie

Un cookie è un piccolo dato salvato nel browser dell'utente.

```
<?php
setcookie("tema", "scuro", time() + 3600);
```

Questo cookie dura un'ora. **3600** sono secondi.

Alla richiesta successiva puoi leggerlo:

```
<?php
$tema = $_COOKIE["tema"] ?? "chiaro";
echo $tema;
```

I cookie sono utili per preferenze semplici, come lingua o tema.

Sessioni

Una sessione salva i dati sul server e collega l'utente a quei dati tramite un identificatore.

Devi chiamare `session_start()` prima di usare `$_SESSION`.

```
<?php
session_start();

$_SESSION["nome"] = "Luca";
```

In un'altra pagina:

```
<?php
session_start();

$nome = $_SESSION["nome"] ?? "ospite";
echo "Ciao $nome";
```

Quando usare le sessioni

Le sessioni sono adatte per login, carrelli e dati temporanei legati alla visita.

I cookie sono adatti per piccole preferenze che possono restare nel browser.

Chiudere una sessione

Per un logout semplice:

```
<?php
session_start();
session_destroy();
```

Attenzione: non salvare password in chiaro dentro cookie o sessioni. Per un login reale servono controlli di sicurezza più completi.

Un esempio di preferenza con cookie

Un cookie è adatto per una preferenza semplice.

```
<?php
setcookie("lingua", "it", time() + 60 * 60 * 24 * 30);
```

Questo cookie dura circa 30 giorni. Alla visita successiva puoi leggere la lingua scelta.

```
<?php
$lingua = $_COOKIE["lingua"] ?? "it";
```

Un esempio di login con sessione

In un login reale dovresti controllare email e password nel database. Qui guardiamo solo il meccanismo della sessione.

```
<?php
session_start();

$_SESSION["utente_id"] = 15;
```

Nelle pagine riservate puoi controllare se l'utente è presente.

```
<?php
session_start();

if (!isset($_SESSION["utente_id"])) {
    echo "Devi accedere";
    exit;
}
```

Errore comune: dimenticare session_start

Se usi `$_SESSION` senza `session_start()`, PHP non apre la sessione. Metti `session_start()` all'inizio del file, prima di stampare HTML.